

guide me to "you deploy a self-contained bundle with all dependencies included"

To deploy your Qt5 application as a **self-contained bundle** on Linux (so that it runs on another robot without system-wide Qt installation), follow one of these methods—**linuxdeployqt** or **cqtdeployer**. Here is a step-by-step guide for each:

Method 1: Using linuxdeployqt

linuxdeployqt automates bundling your app with all Qt libraries, plugins, and dependencies. It can also create a portable AppImage.

Steps

1. Download linuxdeployqt:

- From [GitHub releases](#) or via a package manager.
- Example:

```
wget https://github.com/probonopd/linuxdeployqt/releases/download/continuous/linuxdeployqt-continuous-x86_64.AppImage
chmod +x linuxdeployqt-continuous-x86_64.AppImage
```

2. Prepare your Qt application:

- Make sure your binary (e.g., `robot_ble_communicator`) is compiled in **Release mode**.
- Place the binary and any resources in a folder.

3. Run linuxdeployqt to bundle your application:

- Example:

```
./linuxdeployqt-continuous-x86_64.AppImage ./robot_ble_communicator -appimage
```

- This command generates a self-contained `robot_ble_communicator.AppImage` which you can run directly on other Linux systems^{[1] [2] [3]}.
- **Other useful options:**
 - `-qmake /path/to/qmake`: Specify if your qmake isn't in \$PATH.
 - `-bundle-non-qt-libs`: Bundle non-Qt dependencies too.
 - `-extra-plugins`: Add extra Qt plugins if you need.

Method 2: Using cqtdeployer

cqtdeployer is another cross-platform bundler, also suitable for deploying on both Linux and Windows.

Steps

1. Install cqtdeployer:

- Via Snap:

```
sudo snap install cqtdeployer
sudo snap connect cqtdeployer:process-control
sudo snap connect cqtdeployer:removable-media
sudo snap connect cqtdeployer:system-backup
```

- Or download the latest installer from GitHub [\[4\]](#) [\[5\]](#) [\[6\]](#).

2. Build your project in Release mode.

3. Bundle your binary:

- Example:

```
cqtdeployer -bin robot_ble_communicator
```

- To specify the Qt version or extra libraries:

```
cqtdeployer -bin robot_ble_communicator -qmake /path/to/Qt/5.x.x/gcc_64/bin/qmake
cqtdeployer -bin robot_ble_communicator -libDir /path/to/extra/libs
```

- For QML or additional files, use:

```
cqtdeployer -bin robot_ble_communicator -qmlDir /path/to/qml
```

4. Find the deployment folder:

- Your binary, required Qt libraries, and plugins, plus a launch script, will be included.
- Transfer this folder (or installer) to the target machine [\[4\]](#) [\[5\]](#) [\[7\]](#).

Notes & Tips

- **Always check dependencies:** Run `ldd robot_ble_communicator` to ensure all required libraries are included.
- **AppImage vs folder deployment:** `linuxdeployqt` can generate a single `.AppImage` file, making transfers easy. `cqtdeployer` produces a directory with everything included.
- **Oldest system builds:** To maximize compatibility, consider building and deploying on an *older* Linux system (e.g., Ubuntu 18.04 or older), so your bundle works on more targets [\[8\]](#).

Summary:

For easiest portability, use `linuxdeployqt` or `cqtdeployer` to create a self-contained package or

AppImage. You can then distribute and run your Qt5 application on almost any Linux robot without extra setup. Just copy, mark executable, and run^{[1] [2] [5]}.

*
**

1. <https://discourse.appimage.org/t/linuxdeployqt-new-linux-deployment-tool-for-qt/57>
2. https://blog.csdn.net/xie__jin__cheng/article/details/145142486
3. <https://github.com/probonopd/linuxdeployqt>
4. <https://gist.github.com/Endrll/a96ff2b4d293133067ac2a1f5dad5fe8>
5. <https://github.com/QuasarApp/CQtDeployer/wiki/quickguide>
6. <https://github.com/QuasarApp/CQtDeployer>
7. <https://wiki.qt.io/CQtDeployer>
8. <https://gitee.com/qsaker/linuxdeployqt>